

# 1. Задачи к выполнению

## 1.1. Часть 1

Рассмотрим геометрические фигуры. Общий класс `GeometricObject` используется для моделирования всех геометрических фигур: он содержит свойства цвета `color` и заливки `filled`, а также соответствующие им getter- и setter-методы, свойство `dateCreated` и методы `getDateCreated()` и `toString()`. Метод `toString()` возвращает строковое представление объекта. Классы `Circle` и `Rectangle` порождены от класса `GeometricObject` и наследуют его методы через ключевое слово `extends`.

### 1.1.1. Задача 1

Создайте класс `Triangle` для представления треугольников, который порождается от класса `GeometricObject`. Напишите клиент этих классов — программу, которая запрашивает у пользователя ввести три стороны треугольника, цвет и логическое значение для указания закрашен ли треугольник. Программа должна создавать объект типа `Triangle` с указанными сторонами и присваивать значения свойствам цвет `color` и заливка `isFilled` с помощью этих входных данных. Программа должна отображать площадь `area`, периметр `perimeter`, цвет, а также `true` или `false` для указания, закрашен треугольник или нет.

Класс `Triangle` должен содержать:

- Три поля данных типа `double` с именами `side1`, `side2` и `side3` и с заданными по умолчанию значениями, равными 1.0, для обозначения трёх сторон треугольника.
- Безаргументный конструктор, который создаёт треугольник с заданными по умолчанию значениями.
- Конструктор, который создаёт треугольник с `side1`, `side2` и `side3`.
- Getter-методы для всех трёх полей данных.
- Метод с именем `getArea()`, который возвращает площадь этого треугольника (по формуле Герона).
- Метод с именем `getPerimeter()`, который возвращает периметр этого треугольника.
- Метод с именем `toString()`, который возвращает строковое описание треугольника.

### 1.1.2. Задача 2

В треугольнике сумма длин любых двух сторон больше длины третьей стороны. Класс `Triangle` должен удовлетворять этому правилу. Создайте

класс `IllegalTriangleException` и измените конструктор класса `Triangle`, чтобы выбросить объект типа `IllegalTriangleException`, если треугольник создан со сторонами, нарушающими это правило.

### 1.1.3. Задача 3

Спроектируйте новый класс `Triangle`, который наследуется от абстрактного класса `GeometricObject`.

1. Напишите тестовую программу, которая запрашивает у пользователя ввод трёх сторон треугольника, цвета и логического значения для указания заливки.
2. Программа должна создать объект типа `Triangle` с этими сторонами и задать свойства `color` и `filled`, используя введенные пользователем данные.
3. Программа должна отображать площадь, периметр, цвет и значение `true` или `false` для указания заливки.
4. Реализуйте в классе `GeometricObject` интерфейс `Comparable` и определите статический метод `max()` в классе `GeometricObject` для поиска наибольшего из двух объектов типа `GeometricObject`.
5. Проверьте тестовой программой работу метода `max()` для поиска наибольшего из двух кругов и наибольшего из двух прямоугольников.
6. Вернитесь к классу с именем `ComparableCircle`, который наследуется от `Circle` и реализует `Comparable`. Напишите тестовую программу, чтобы найти наибольший из двух экземпляров класса `ComparableCircle` и наибольший между кругом и прямоугольником, используя метод `compareTo()`.

### 1.1.4. Задача 4

1. Создайте интерфейс с именем `Colorable` с помощью метода `howToColor()` типа `void`. Каждый класс раскрашиваемого объекта должен реализовывать интерфейс `Colorable`.
2. Спроектируйте класс `Square`, который наследуется от `GeometricObject` и реализует `Colorable`. Реализуйте метод `howToColor()` для отображения сообщения: Раскрасьте все четыре стороны.
3. Класс `Square` содержит поле данных `side` с getter- и setter-методами, а также конструктор для создания `Square` с указанной стороной. У класса `Square` есть скрытое поле данных типа `double` с именем `side` и getter- и setter-методами. У него есть безаргументный конструктор,

который создаёт объект типа `Square` со стороной, равной 0, и ещё один конструктор, который создаёт объект типа `Square` с указанной стороной.

4. Напишите тестовую программу, которая создаёт массив из пяти объектов типа `GeometricObject`. Для каждого объекта в массиве отобразите его площадь и вызовите метод `howToColor()`, если его можно раскрасить.

#### 1.1.4.1. Решение

```
1 // geometry/GeometricObject.java
2 package ru.haykor.mirea.geometry;
3
4 public abstract class GeometricObject implements
Comparable<GeometricObject> {
5     private String color = "white";
6     private boolean filled;
7     private final java.util.Date dateCreated;
8
9     protected GeometricObject() {
10         dateCreated = new java.util.Date();
11     }
12
13     protected GeometricObject(String color, boolean filled) {
14         dateCreated = new java.util.Date();
15         this.color = color;
16         this.filled = filled;
17     }
18
19     public String getColor() {
20         return color;
21     }
22
23     public void setColor(String color) {
24         this.color = color;
25     }
26
27     public boolean isFilled() {
28         return filled;
29     }
30
31     public void setFilled(boolean filled) {
32         this.filled = filled;
```

```

33     }
34
35     public java.util.Date getDateCreated() {
36         return dateCreated;
37     }
38
39     public abstract double getArea();
40
41     @Override
42     public int compareTo(GeometricObject o) {
43         return Double.compare(getArea(), o.getArea());
44     }
45
46     public static GeometricObject max(GeometricObject o1,
47     GeometricObject o2) {
48         return (o1.compareTo(o2) ≥ 0) ? o1 : o2;
49     }
50
51     @Override
52     public String toString() {
53         return "created on " + dateCreated + "\ncolor: " +
54     color + " and filled: " + filled;
55     }
56 }

```

```

1 // geometry/Circle.java
2 package ru.haykor.mirea.geometry;
3
4 public class Circle extends GeometricObject {
5     private double radius;
6
7     public Circle() {
8     }
9
10    public Circle(double radius) {
11        this.radius = radius;
12    }
13
14    public Circle(double radius, String color, boolean filled)
15    {
16        super(color, filled);
17        this.radius = radius;
18    }
19 }

```

```

18
19     public double getRadius() {
20         return radius;
21     }
22
23     public void setRadius(double radius) {
24         this.radius = radius;
25     }
26
27     public double getDiameter() {
28         return 2 * radius;
29     }
30
31     public double getPerimeter() {
32         return 2 * radius * Math.PI;
33     }
34
35     @Override
36     public double getArea() {
37         return radius * radius * Math.PI;
38     }
39
40     public void printCircle() {
41         System.out.println("The circle is created " +
getDateCreated() + " and the radius is " + radius);
42     }
43 }

```

```

1 // geometry/Rectangle.java
2 package ru.haykor.mirea.geometry;
3
4 public class Rectangle extends GeometricObject {
5     private double width;
6     private double height;
7
8     public Rectangle() {
9     }
10
11     public Rectangle(double width, double height) {
12         this.width = width;
13         this.height = height;
14     }
15

```

```

16     public Rectangle(double width, double height, String
color, boolean filled) {
17         super(color, filled);
18         this.width = width;
19         this.height = height;
20     }
21
22     public double getWidth() {
23         return width;
24     }
25
26     public void setWidth(double width) {
27         this.width = width;
28     }
29
30     public double getHeight() {
31         return height;
32     }
33
34     public void setHeight(double height) {
35         this.height = height;
36     }
37
38     public double getPerimeter() {
39         return 2 * (width + height);
40     }
41
42     @Override
43     public double getArea() {
44         return width * height;
45     }
46 }

```

```

1 // geometry/IllegalTriangleException.java
2 package ru.haykor.mirea.geometry;
3
4 public class IllegalTriangleException extends Exception {
5     public IllegalTriangleException(String message) {
6         super(message);
7     }
8 }

```

```

1 // geometry/Triangle.java
2 package ru.haykor.mirea.geometry;
3
4 public class Triangle extends GeometricObject {
5     private double side1 = 1.0;
6     private double side2 = 1.0;
7     private double side3 = 1.0;
8
9     public Triangle() {
10    }
11
12    public Triangle(double side1, double side2, double side3)
13    throws IllegalArgumentException {
14        if (side1 + side2 ≤ side3 || side1 + side3 ≤ side2
15        || side2 + side3 ≤ side1) {
16            throw new IllegalArgumentException(
17                "Сумма длин любых двух сторон должна быть
18                больше длины третьей стороны");
19        }
20
21        this.side1 = side1;
22        this.side2 = side2;
23        this.side3 = side3;
24    }
25
26    public double getSide1() {
27        return side1;
28    }
29
30    public double getSide2() {
31        return side2;
32    }
33
34    public double getSide3() {
35        return side3;
36    }
37
38    public double getPerimeter() {
39        return side1 + side2 + side3;
40    }
41
42    @Override
43    public double getArea() {

```

```

41         double s = (side1 + side2 + side3) / 2;
42         return Math.sqrt(s * (s - side1) * (s - side2) * (s -
side3));
43     }
44
45     @Override
46     public String toString() {
47         return "Треугольник: сторона1 = " + side1 + " сторона2
= " + side2 + " сторона3 = " + side3;
48     }
49 }

```

```

1 // app/TestTriangle.java
2 package ru.haykor.mirea.app;
3
4 import java.util.Scanner;
5
6 import ru.haykor.mirea.geometry.IllegalTriangleException;
7 import ru.haykor.mirea.geometry.Triangle;
8
9 public class TestTriangle {
10     public static void main(String[] args) {
11         Scanner scanner = new Scanner(System.in);
12
13         System.out.print("Введите три стороны треугольника:
");
14         double side1 = scanner.nextDouble();
15         double side2 = scanner.nextDouble();
16         double side3 = scanner.nextDouble();
17
18         System.out.print("Введите цвет: ");
19         String color = scanner.next();
20
21         System.out.print("Треугольник закрашен? (true/false):
");
22         boolean filled = scanner.nextBoolean();
23
24         try {
25             Triangle triangle = new Triangle(side1, side2,
side3);
26             triangle.setColor(color);
27             triangle.setFilled(filled);
28

```

```

29         System.out.println(triangle);
30         System.out.println("Площадь: " +
triangle.getArea());
31         System.out.println("Периметр: " +
triangle.getPerimeter());
32         System.out.println("Цвет: " +
triangle.getColor());
33         System.out.println("Закрашен: " +
triangle.isFilled());
34     } catch (IllegalArgumentException e) {
35         System.out.println("Ошибка: " + e.getMessage());
36     }
37
38     scanner.close();
39 }
40 }

```

```

1 // geometry/ComparableCircle.java
2 package ru.haykor.mirea.geometry;
3
4 public class ComparableCircle extends Circle {
5
6     public ComparableCircle(double radius) {
7         super(radius);
8     }
9
10    @Override
11    public int compareTo(GeometricObject o) {
12        Circle other = (Circle) o;
13        return Double.compare(getRadius(), other.getRadius());
14    }
15 }

```

`ComparableCircle` наследует реализацию `Comparable<GeometricObject>` от `GeometricObject` (повторно реализовать интерфейс с другим параметром типа в Java нельзя), поэтому вместо этого переопределяется сам метод `compareTo()`, чтобы сравнивать круги по радиусу, а не по площади, как это делает родительский класс.

```

1 // app/TestMax.java
2 package ru.haykor.mirea.app;

```

```

3
4 import ru.haykor.mirea.geometry.Circle;
5 import ru.haykor.mirea.geometry.ComparableCircle;
6 import ru.haykor.mirea.geometry.GeometricObject;
7 import ru.haykor.mirea.geometry.Rectangle;
8
9 public class TestMax {
10     public static void main(String[] args) {
11         Circle circle1 = new Circle(4);
12         Circle circle2 = new Circle(5);
13         GeometricObject maxCircle =
GeometricObject.max(circle1, circle2);
14         System.out.println("Наибольший круг имеет площадь " +
maxCircle.getArea());
15
16         Rectangle rectangle1 = new Rectangle(2, 3);
17         Rectangle rectangle2 = new Rectangle(4, 1);
18         GeometricObject maxRectangle =
GeometricObject.max(rectangle1, rectangle2);
19         System.out.println("Наибольший прямоугольник имеет
площадь " + maxRectangle.getArea());
20
21         ComparableCircle comparableCircle1 = new
ComparableCircle(4);
22         ComparableCircle comparableCircle2 = new
ComparableCircle(5);
23         ComparableCircle maxComparableCircle =
24             comparableCircle1.compareTo(comparableCircle2)
≥ 0 ? comparableCircle1 : comparableCircle2;
25         System.out.println("Наибольший ComparableCircle имеет
радиус " + maxComparableCircle.getRadius());
26
27         GeometricObject maxBetweenCircleAndRectangle =
GeometricObject.max(circle1, rectangle1);
28         System.out.println("Наибольший между кругом и
прямоугольником имеет площадь "
29             + maxBetweenCircleAndRectangle.getArea());
30     }
31 }

```

```

1 // geometry/Colorable.java
2 package ru.haykor.mirea.geometry;
3

```

```
4 public interface Colorable {
5     void howToColor();
6 }
```

```
1 // geometry/Square.java
2 package ru.haykor.mirea.geometry;
3
4 public class Square extends GeometricObject implements
Colorable {
5     private double side;
6
7     public Square() {
8     }
9
10    public Square(double side) {
11        this.side = side;
12    }
13
14    public double getSide() {
15        return side;
16    }
17
18    public void setSide(double side) {
19        this.side = side;
20    }
21
22    @Override
23    public double getArea() {
24        return side * side;
25    }
26
27    @Override
28    public void howToColor() {
29        System.out.println("Раскрасьте все четыре стороны");
30    }
31 }
```

```
1 // app/TestColorable.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.geometry.Circle;
5 import ru.haykor.mirea.geometry.Colorable;
```

```

6 import ru.haykor.mirea.geometry.GeometricObject;
7 import ru.haykor.mirea.geometry.Rectangle;
8 import ru.haykor.mirea.geometry.Square;
9
10 public class TestColorable {
11     public static void main(String[] args) {
12         GeometricObject[] objects = {
13             new Circle(3),
14             new Rectangle(2, 5),
15             new Square(4),
16             new Square(2.5),
17             new Circle(1.5)
18         };
19
20         for (GeometricObject object : objects) {
21             System.out.println("Площадь: " +
object.getArea());
22
23             if (object instanceof Colorable colorable) {
24                 colorable.howToColor();
25             }
26         }
27     }
28 }

```

## 1.2. Часть 2

Обработка исключений в Java позволяет поддерживать нормальный поток работы приложения при возникновении ошибок времени выполнения. `RuntimeException`, `Error` и их подклассы называются непроверяемыми (unchecked) исключениями и не требуют обязательной обработки, тогда как `ArrayIndexOutOfBoundsException` — проверяемое исключение, которое удобно использовать для отбраковки недопустимого пользовательского ввода без явной валидации границ массива.

### 1.2.1. Задача 1

С помощью двух массивов напишите программу, которая предложит пользователю ввести целое число от 1 до 12, а затем отобразит месяц и количество дней, соответствующие этому целому числу. Если пользователь вводит недопустимое число, то программа должна отображать «Недопустимое число» с помощью перехвата `ArrayIndexOutOfBoundsException`. Программа должна

быть реализована таким образом, чтобы предотвратить ввод пользователем любого числа, кроме целого.

```
1 String[] months = {"январь", "февраль", "март", "апрель",  
2   "май",  
3   "июнь", "июль", "август", "сентябрь", "октябрь", "ноябрь",  
   "декабрь"};  
4 int[] dom = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

### 1.2.1.1. Решение

```
1 // app/MonthDays.java  
2 package ru.haykor.mirea.app;  
3  
4 import java.util.InputMismatchException;  
5 import java.util.Scanner;  
6  
7 public class MonthDays {  
8     public static void main(String[] args) {  
9         String[] months = {"январь", "февраль", "март",  
10        "апрель", "май",  
11        "июнь", "июль", "август", "сентябрь",  
12        "октябрь", "ноябрь", "декабрь"};  
13        int[] dom = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31,  
14        30, 31};  
15  
16        Scanner scanner = new Scanner(System.in);  
17  
18        try {  
19            System.out.print("Введите номер месяца (1-12): ");  
20            int month = scanner.nextInt();  
21  
22            System.out.println("Месяц: " + months[month - 1] +  
23            ", количество дней: " + dom[month - 1]);  
24        } catch (ArrayIndexOutOfBoundsException e) {  
25            System.out.println("Недопустимое число");  
26        } catch (InputMismatchException e) {  
27            System.out.println("Нужно ввести целое число");  
28        }  
29        scanner.close();  
30    }  
31 }
```

### 1.2.2. Задача 2

Дополните код 1 задания данной практической работы таким образом, чтобы при выборе февраля была возможность записать год. Добавьте в программу метод для расчёта является ли год високосным и измените вывод для количества дней в феврале.

#### 1.2.2.1. Решение

```
1 // app/MonthDaysLeapYear.java
2 package ru.haykor.mirea.app;
3
4 import java.util.InputMismatchException;
5 import java.util.Scanner;
6
7 public class MonthDaysLeapYear {
8     public static void main(String[] args) {
9         String[] months = {"январь", "февраль", "март",
10 "апрель", "май",
11 "июнь", "июль", "август", "сентябрь",
12 "октябрь", "ноябрь", "декабрь"};
13         int[] dom = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
14 30, 31};
15
16         Scanner scanner = new Scanner(System.in);
17
18         try {
19             System.out.print("Введите номер месяца (1-12): ");
20             int month = scanner.nextInt();
21
22             System.out.print("Введите год: ");
23             int year = scanner.nextInt();
24
25             int days = dom[month - 1];
26             if (month == 2 && isLeapYear(year)) {
27                 days = 29;
28             }
29
30             System.out.println("Месяц: " + months[month - 1] +
31 " , количество дней: " + days);
32         } catch (ArrayIndexOutOfBoundsException e) {
33             System.out.println("Недопустимое число");
34         } catch (InputMismatchException e) {
35             System.out.println("Нужно ввести целое число");
36         }
37     }
38 }
```

```
32     }
33
34     scanner.close();
35 }
36
37 private static boolean isLeapYear(int year) {
38     return (year % 4 == 0 && year % 100 != 0) || year %
400 == 0;
39 }
40 }
```